

MDS290 Python Reference Guide

Print statements

Python's equivalent to `fprintf` or `disp` is `print`.

MATLAB	Python
<code>fprintf("Hello, World!\n");</code>	<code>print("Hello, World!")</code>
Output:	Output:
Hello, World!	Hello, World!

Modern Python uses “f-strings” to insert variables into print statements. In f-strings, place an “f” before the string quotes, and use curly brackets { } to place the variable in the string:

MATLAB	Python
<code>xval = 1.2;</code> <code>fprintf("x = %.2f\n", xval);</code>	<code>xval = 1.2</code> <code>print(f"x = {xval:.2f}")</code>
Output:	Output:
1.20	1.20

0 vs 1 indexing

MATLAB vectors and matrices are indexed from 1. In Python, they are indexed from 0.

MATLAB	Python
<code>a = [3, 4, 5];</code> <code>% Print the first element of a</code> <code>fprintf("%.0f\n", a(1));</code>	<code>a = [3, 4, 5]</code> <code># Print the first element of a</code> <code>print(a[0])</code>
Output:	Output:
3	3

It is important to note that the ‘a’ variable created in Python above is not actually a *vector* – it is called a “list”, and lists can hold different types of variables (e.g., `a = ["one", "two"]`).

Python does not have built-in mathematical vectors or matrices like MATLAB. We will have to import a library to create such vectors and matrices, which we will return to later.

Creating Matrices and Vectors

Python does not have matrix algebra capabilities right away. It is common to import a library called numpy (Numerical Python) to create and use vectors and matrices.

MATLAB	Python
<pre>a = [2, 4, 6]; disp(a); M = [1, 2, 3; 4, 5, 6; 7, 8, 9]; disp(M);</pre>	<pre>import numpy as np a = np.array([2, 4, 6]) print(a) M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) print(M)</pre>

Once you import a library, you do not have to import it again in that code. It is standard practice to have all “import” commands occur at the top of Python code.

Subsampling vectors and matrices (“slicing”)

If you want to access some components of a vector or matrix, you can use the colon operator (:) as in MATLAB, with the same (row, column) ordering.

MATLAB	Python
<pre>M = [1, 2, 3; 4, 5, 6; 7, 8, 9]; % Print first row disp(M(1,:)); % Print last column disp(a(:,3));</pre>	<pre>import numpy as np M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Print first row print(M[0,:]) # Print last column print(M[:,2])</pre>

Creating figures

Python does not have strong plotting capabilities right away. It is common to import a library called **pyplot** from another library called **matplotlib**.

MATLAB	Python
<pre>x = linspace(0, 10, 100); y_squared = x.^2; y_cubed = x.^3; figure; plot(x,y_squared,DisplayName="squared"); hold on; plot(x, y_cubed,DisplayName="cubed"); hold off; xlabel('x'); ylabel('y'); legend; % Save figure to file [optional] % exportgraphics(gcf, "figure.png");</pre>	<pre>import numpy as np import matplotlib.pyplot as plt x = np.linspace(0, 10, 100) y_squared = x**2 y_cubed = x**3 plt.plot(x, y_squared, label='squared') plt.plot(x, y_cubed, label='cubed') plt.xlabel('x') plt.ylabel('y') plt.legend() # Save figure to file [optional] # plt.savefig("figure.png") # Show figure plt.show() plt.close()</pre>

You can also create subplots:

MATLAB	Python
<pre>x = linspace(0, 10, 100); y_squared = x.^2; y_cubed = x.^3; figure; subplot(1,2,1); plot(x, y_squared, 'r'); title('Squared'); xlabel('x'); ylabel('y'); subplot(1,2,2); plot(x, y_cubed, 'b'); title('Cubed'); xlabel('x'); ylabel('y');</pre>	<pre>x = np.linspace(0, 10, 100) y_squared = x**2 y_cubed = x**3 plt.figure(figsize=(10, 4)) plt.subplot(1, 2, 1) plt.plot(x, x**2, color='r') plt.title('Squared'); plt.xlabel('x'); plt.ylabel('y'); plt.subplot(1, 2, 2) plt.plot(x, x**3, color='b') plt.title('Cubed'); plt.xlabel('x'); plt.ylabel('y'); plt.tight_layout() plt.show()</pre>

For and While Loops

Python for- and while-loops do not close loops with “end” the way MATLAB does.

Instead, Python uses **indentation** (typically **4 spaces**, or 1 tab) to define where a loop ends. They can also have more complex notation to choose how the loop run. Note that in the following Python code, `i` will iterate from 0 to 9 (not to 10).

MATLAB	Python
<pre>a = 1; for i = 1:10 a = a + 1; b = a * a; end</pre>	<pre>a = 1 for i in range(10): a = a + 1 b = a * a print(f'i={i}, a={a}')</pre>

Start/stop/increment values can also be used. If two values are used, it is `range(start, stop)` but remember that the Python loop will go up to, but not include, the last value:

MATLAB	Python
<pre>a = 1; for i = 2:5 a = a + 1; b = a * a; fprintf('i=%d, a=%.0f\n', i, a); end</pre>	<pre>a = 1 for i in range(2, 6): a = a + 1 b = a * a print(f'i={i}, a={a}')</pre>

You can also set an increment using `range(start, stop, increment)`:

MATLAB	Python
<pre>a = 1; for i = 2:2:10 a = a + 1; b = a * a; fprintf('i=%d, a=%.0f\n', i, a); end</pre>	<pre>a = 1 for i in range(2, 11, 2): a = a + 1 b = a * a print(f'i={i}, a={a}')</pre>

If/Else

In Python, if/else conditionals follow the same indentation rules as loops. You do not need parentheses around conditions, but you must include a colon (:) at the end of each condition. Where MATLAB has “`elseif`”, Python uses “`elif`”:

MATLAB	Python
<pre>x = 10; if x > 15 fprintf("Greater than 15\n"); elseif x == 10 fprintf("Exactly 10\n"); else fprintf("Something else\n"); end</pre>	<pre>x = 10 if x > 15: print("Greater than 15") elif x == 10: print("Exactly 10") else: print("Something else")</pre>

Python uses literal words “`and`” instead of `&&`, and “`or`” instead of `||`:

MATLAB	Python
<pre>age = 20; student = true; if age < 18 student fprintf("Discount!\n"); elseif student && age > 60 fprintf("Special!\n"); else fprintf("Full Price\n"); end</pre>	<pre>age = 20 student = True if age < 18 or student: print("Discount!") elif student and age > 60: print("Special!") else: print("Full Price")</pre>

Functions

In Python, functions are created using `def`, and output values with `return`. Like other Python syntax, functions require indentation (typically 4 spaces, or 1 tab):

MATLAB	Python
<pre>function [y] = func_demo(a, b) y = 2 * a + b; end</pre>	<pre>def func_demo(a, b): y = 2 * a + b return y</pre>

You would then call these functions in a similar way as in MATLAB:

MATLAB	Python
<pre>z = func_demo(1, 2) fprintf("z=%d\n", z);</pre>	<pre>z = func_demo(1, 2) print(f"z={z}")</pre>

Functions can have multiple outputs:

MATLAB	Python
<pre>function [p,q] = func_demo2(a, b) p = 2 * a + b; q = 2 * a; end</pre>	<pre>def func_demo2(a, b): p = 2 * a + b q = 2 * a return p,q</pre>

MATLAB returns multiple outputs in brackets, but Python can return them separately:

MATLAB	Python
<pre>[y, z] = func_demo(1, 2) fprintf("y=%d, z=%d\n", y, z);</pre>	<pre>y,z = func_demo2(1, 2) print(f" y = {y}, z = {z}")</pre>